

零开销抽象

Zero-Overhead Abstraction:把代价从运行时挪走——编译期多态、编译期计算、预分配

Robert (Bo) Hou · Robert's Technical Notes · 2026 年 6 月

摘要。C++ 的立身之本是一句承诺:“你不用的东西不付代价;你用的东西,手写也不会更快。”这就是零开销抽象(zero-overhead abstraction)。本报告用三个主题逐一验证这条承诺——谁违反了它,又如何恢复它。虚函数违反了它(运行时多态挡住内联),CRTP

用编译期多态恢复它;老式模板元编程能编译期计算但语法痛苦,constexpr 用普通函数语法恢复它;malloc 在热路径上代价高昂,arena/pool 分配器用预分配把分配退化成一次指针操作。贯穿三题的是同一条主线:把代价从运行时挪走——挪到编译期(CRTP、constexpr),或挪到启动时(预分配)。这也正是前两座山(正确性、速度)的结论在“代码抽象”一侧的落地:数据要连续、可预测、可复用,而抽象要在编译期或启动时蒸发,不在热路径上留下运行时代价。

0. 零开销抽象:一句承诺

Bjarne Stroustrup 对 C++ 的定义里有一条核心承诺,值得先钉在这里,因为本报告三题都是在验证它:

零开销抽象(zero-overhead abstraction)。其一,你不用的特性,不付代价(what you don't use, you don't pay for)。其二,你用到的抽象,手写同样的东西也不会更快(what you do use, you couldn't hand-code any better)。这条承诺是 C++ 相对更高级语言的立身之本:它让你既能写出高层抽象(接口、泛型、容器),又不必为此付出运行时税。本报告逐一检验:虚函数在哪里违反了这条承诺,而 CRTP / constexpr / 预分配分配器又如何恢复它。

1. 虚函数 vs CRTP:编译期多态恢复内联

一个基类接口、一个派生实现,热循环里调用上亿次。先厘清一个常见误解:很多人说“虚函数慢是因为每次 cache miss”——错。在热循环里,对象、vtable 指针、vtable、目标函数地址都被反复访问,第一次之后全在 cache 里命中,不会每次 miss。同理,间接跳转 call [reg] 的目标地址也能被间接分支预测器学会:只要 p 指向的类型稳定,预测器一次学会后就一直猜准,跳转 miss 也是最多几次。

虚函数真正的代价:挡住内联,而非跳转或 cache。一个虚调用比普通调用多三步:读对象头部的 vtable 指针、从 vtable 读目标函数地址、间接跳转过去。前两步是 cache 命中(开销小),间接跳转在类型稳定时也几乎无惩罚。真正稳定存在的代价是:编译期不知道 p 的具体类型,所以编译器不敢内联。而对一个小热函数(如 return x*2+1;),函数体本身比“一次完整的函数调用”还小;能内联时它几乎免费、还能与循环一起向量化/常量折叠,不能内联时则每次付全套调用开销、且丧失所有跨函数优化。开销不在跳转、不在缓存,在“该内联的没内联”。

CRTP(奇异递归模板模式)正是来恢复内联的。把基类做成模板、派生类型作为模板参数传入,基类在编译期就知道具体类型,从而能内联:

```

template <class Derived>
struct Base {
    int compute(int x) const {
        return static_cast<const Derived*>(this)->compute_impl(x);
    }
    // Derived is a template param -> known at compile time -> inlinable
};
struct Impl : Base<Impl> { // the curious part: the class passes itself to its base template
    int compute_impl(int x) const { return x * 2 + 1; }
};

```

为何不无限递归? `struct Impl : Base` 看似“定义 `Impl` 要先有 `Impl`”,但实例化 `Base` 只需 `Impl` 的名字(不完整类型,因基类只通过 `Derived*` 指针使用它,指针不需完整定义),而基类成员函数体延迟到调用时才实例化(那时 `Impl` 已完整)。这与 `struct Node { Node* next; };` 自引用却不递归是同一道理——指针打破了递归。仅当基类放 `Derived` member(完整对象而非指针)时才会真出错。

实测:代价不在“那一次跳转”,而在“保住编译器的优化视野”。对小热函数 $x*2+1$ 调用数亿次,虚函数 vs CRTP:在无法被整体优化掉的构造下(如带依赖链),虚 vs CRTP $\approx 1.27\times$ ——这是“间接跳转 + 无法内联的调用开销”的直接代价。而在可向量化/可整体消除的构造下,CRTP/direct 被编译器整个循环优化没(看穿累加式、用闭式或向量化),虚函数只能硬跑数亿次间接调用——比值趋于无穷。这才是真正的一课:虚函数的代价不是省下那一次跳转,而是它堵死了编译器的内联与所有跨函数优化;一旦能内联,编译器能做的优化是无上限的。(方法论提醒:微基准极易被污染——依赖链、指令调度、被优化消除都会让数字骗人,须谨慎构造。)

权衡:CRTP 拿“运行时灵活性”换“零开销 + 可内联”。它失去了运行时多态——不同的 `Base` 与 `Base` 是不同类型,不能装进同一个容器。故 CRTP 适合编译期即确定类型的性能热路径;需要运行时才决定类型并混装同一容器的场景,仍用虚函数。

2. constexpr:把计算搬到编译期

CRTP 是编译期确定类型,constexpr 是编译期完成计算。历史上“编译期计算”有两条老路,都难用:宏(预处理文本替换,不懂类型/语义,且预处理器禁止宏自我递归,还有优先级、参数多次求值、分号吞并等经典陷阱)与模板元编程(用递归模板实例化模拟计算,图灵完备——1994 年 Unruh 曾用它在编译期算质数——但语法极痛苦,且把值编码成类型、编译慢、报错爆炸)。constexpr(C++11 起)用普通函数语法统一取代二者。

constexpr 的核心性质:同一份代码,编译期/运行时两用。 `constexpr int fib(int n){ if(n<2) return n; return fib(n-1)+fib(n-2); }` 这一个函数:当结果被要求为常量时(如 `constexpr int a = fib(40);`),强制在编译期算出,运行时直接是常数 102334155,运行开销为零;当参数是运行时才知道的值时(如 `int b = fib(m);`, `m` 来自输入),它退化为普通运行时函数。同一份代码覆盖两种执行时机,视上下文而定——这正是 constexpr 相对模板元编程的根本优势:后者只能编译期(Fib 中 `m` 必须是编译期常量,写不了运行时版)。

其余优势:constexpr 用正常 `if/for/while`、局部变量写逻辑,可读可调试,不像模板元编程要用特化模拟条件、递归模拟循环;它是值就是值,不把值编码成一堆类型,编译更快、报错正常。其能力随标准持续增强:C++14 放开函数体,C++17 有 `if constexpr`(编译期分支),C++20 甚至允许 `constexpr` 中用 `new`、虚函数、`std::vector`。如今绝大多数编译期计算都用 `constexpr`,模板元编程退居真正需要“类型层面操作”的场景。

三个 `const*` 关键词(高频面试题)。 `constexpr`:可以在编译期算(上下文要求且参数为常量时),但允许退化到运行时。`constexpr(C++20)`:必须编译期算,传运行时变量即编译报错(“强制编译期”版)。`constexpr(C++20)`:保证变量在编译期完成初始化(解决静态初始化顺序问题),但变量本身不必是常量。一句话:`constexpr` = 能编译期(可退化);`constexpr` = 必须编译期;`constexpr` = 保证编译期初始化。

3. 自定义分配器:热路径不 new

这一题是“机制内存学”那座山结论的落地:热路径别 `new`。先看 `malloc` 到底慢在哪。

`malloc` 的代价 + 它带来的间接伤害。一次 `malloc` 要:在空闲块链表(free list,通常双向链表)中遍历查找一块合适大小的块;记账(每块 header 记大小,释放时合并相邻空闲块防碎片);多线程下加锁(堆是共享的);堆不够时向 OS 要内存,触发 `mmap/sbrk` 系统调用(陷入内核,极慢)。而“从预分配数组拿一个槽”只是移动一个指针,差几十到几百倍。间接伤害:`new` 出来的对象散落在堆各处,之后遍历它们(尤其链表)就是 pointer chasing → cache miss + prefetch 失效(见“机制内存学”)。`malloc` 不仅分配时慢,分配出来的东西之后访问也慢。

3.1 Arena(bump / 线性分配器)

预先 `malloc` 一大块连续内存,维护一个指向“下一个空闲位置”的指针 `top`。分配 `= top += size;` `return old_top`——就一个指针加法, $O(1)$,不遍历、不加锁、不 `syscall`。对象大小可任意(都往后 bump)。代价:不支持单独释放——`top` 只能往前,释放是整块 reset 回起点。这正是它快的原因:放弃“单独释放”→ 不用 `free` list、不用记账 → 分配退化成一个加法。适合“一批对象生命周期一致、用完一起扔”的场景(如处理一个请求/一帧后整体

reset)。

3.2 Pool(对象池 / 固定大小分配器)

若需频繁分配/释放同样大小的对象(如订单节点),用

pool:预分配一大块,切成固定大小的槽。因所有槽等大,第 i 个槽地址 $= \text{base} + i * \text{slot_size}$,可直接计算,寻址规整;空闲槽用 `free list` 串起,分配摘链表头、释放插回链表头,皆 $O(1)$ ——因所有槽等大,拿哪个都行,不用“找合适的”、不用合并碎片,这正是它比 `malloc` 的 `free list` 快的原因(`malloc` 要处理任意大小,得找、得合并)。

Arena vs Pool:同一预分配思想的两种取舍。两者共性:预分配一大块连续内存(cache 友好、可 prefetch), $O(1)$ 分配,热路径不碰 `malloc`。区别:arena 把连续空间当“一条往前推的带子”,任意大小、bump 分配、不支持单独释放(整块 reset);pool 把连续空间切成等大的格子,规整寻址($\text{base} + i * \text{size}$)、可单独释放并复用(任何空槽等价)。简言之:pool 用“固定大小”换来“单独释放 + 规整寻址 + 无脑复用”;arena 用“放弃单独释放”换来“任意大小 + 极致简单(就一个 bump)”。HFT 两者都用:一批同生命周期的临时对象用完齐扔 $\rightarrow$ arena;频繁 new/delete 的同类对象 $\rightarrow$ pool。

3.3 PMR:标准库的落地(C++17)

C++17 的 `std::pmr`(polymorphic memory resource)让标准容器可以换掉分配器,不必手写:`std::pmr::monotonic_buffer_resource` 是标准库版的 arena,`std::pmr::unsynchronized_pool_resource` 是标准库版的 pool。写 `std::pmr::vector v{&arena;};`,这个 vector 便从你的 arena 分配、不碰 `malloc`。面试高频:“怎么让 `std::vector` 不用 `malloc`?”——PMR + `monotonic_buffer_resource`。

4. 综述:三题一条线,三座山一个结论

零开销抽象的三题,主题统一为把代价从运行时挪走:CRTP 把“类型决议”挪到编译期(恢复内联),constexpr 把“计算”挪到编译期(运行时白拿结果),arena/pool 把“分配”挪到启动时(热路径只剩指针操作)。而这三题又与前两座山的结论在同一处汇合:

三座山的共同工程结论。无锁(正确性)推出:数组槽位复用、从不 delete,绕过安全内存回收、免疫 ABA。机制内存学(速度)推出:数组地址可预测,prefetch/MLP 能翻越内存墙;链表 pointer chasing 让延迟裸露。零开销抽象(本报告)推出:预分配 + $O(1)$ 分配让热路径不碰 `malloc`,连续布局让对象 cache 友好,而 CRTP/constexpr 让抽象在编译期蒸发。三条线指向同一句话:能连续、能预测、能复用,就别动态分配、别指针追逐、别把可以在编译期或启动时确定的代价留到运行时。这就是高性能 C++ 的底层世界观——抽象是免费的,只要你把代价挪对地方。